

CS-1002 Programming Fundamentals Spring-2026

ASSIGNMENT-02

Instructions:

Dear students' failure to submit according to the below format would result in zero marks in the relevant evaluation.

- i. For each question in your assignment, make a separate cpp file e.g. for question 1, make ROLL-NUM_SECTION_Q#.cpp (24i-0001_Q1.cpp) and so on. Each file that you submit must contain your name, student-id, and assignment # on top of the file in comments.
- ii. Combine all your work in one folder. The folder must contain only .cpp files (no binaries, no exe files etc.).
- iii. Run and test your program on a lab machine before submission.
- iv. Rename the folder as ROLL-NUM_SECTION (e.g. 24i-0001) and compress the folder as a zip file. (e.g. 24i-0001.zip). do not submit .rar file.
- v. Submit the .zip file on Google Classroom within the deadline.
- vi. Submission other than Google classroom (e.g. email etc.) will not be accepted.
- vii. The student is solely responsible to check the final zip files for issues like corrupt file, virus in the file, mistakenly exe sent. If we cannot download the file from Google classroom due to any reason it will lead to zero marks in the assignment.
- viii. Displayed output should be well mannered and well presented. Use appropriate comments and indentation in your source code.
- ix. Total Marks: 90.**
- x. If there is a syntax error in code, zero marks will be awarded in that part of assignment.
- xi. Your code must be generic.

Late Submission policy will be applied as described in course outline.

Plagiarism: Plagiarism is not allowed. If found plagiarized, you will be awarded zero marks in the assignment (copying from the internet is the easiest way to get caught).

Note: Follow the instruction given to the letter, failing to do so will result in a zero.

Deadline to submit assignment is 22nd March 2026 11:59 PM.

Question 1

Sara wants to develop tools for her team at the tech company. Her team needs a tool that converts numbers from binary (base 2) or octal (base 8) into decimal (base 10). However, the input numbers in binary or octal will always be 3, 4, or 5 digits long to simplify their tasks, the system would handle all 3 cases itself and we don't want to burden the user by making them input the number of digits as well.

Your job is to help Sara by writing a C++ program that will prompt the user to input a binary or octal number (3, 4, or 5 digits long), and the program will convert that number into decimal.

Requirements:

- The user will first choose whether they are inputting a binary (base 2) or octal (base 8) number:
 - Press **1** to enter a binary number.
 - Press **2** to enter an octal number.
 - Based on the user's selection, prompt them to input a number with either **3, 4, or 5 digits**.
 - For binary, the number must only contain **0 and 1**.
 - For octal, the number must only contain digits from **0 to 7**.
 - If the user inputs a number that isn't **3, 4, or 5 digits long** it should display an error message and not produce any other output.
 - Use a **switch statement** to handle the user's selection of either binary or octal.
 - Convert the valid input number to **decimal (base 10)** and display the result.
 - If the user enters any choice other than **1 or 2** for the base selection, display an **error message** and terminate the program.
-

Question 2

FESCO Residential Electricity Bill Calculation

You are tasked with developing a program that calculates the **monthly electricity bill for residential customers of FESCO (Federal Electric Supply Company)** based on their

monthly consumption in units. The calculation differs based on the number of units consumed, and it includes applicable taxes.

Requirements

1. The user will input the following:
 - The number of **units consumed in a month** (should be a **non-negative integer**).
2. The bill calculation will follow these rules:

For consumption of:

- **0 to 100 units: PKR 5 per unit**
 - **101 to 300 units: PKR 7 per unit**
 - **301 to 400 units: PKR 24.40 per unit**
 - **401 to 500 units: PKR 24.91 per unit**
 - **501 to 600 units: PKR 26.15 per unit**
 - **601 to 700 units: PKR 27.59 per unit**
3. Taxes:
 - The total bill will incur an additional **10% tax** (for general tax).
 4. The program should ensure:
 - **Input validation** to ensure the units consumed is a **non-negative integer**.
 5. The program should calculate:
 - **Total bill before tax**
 - **Tax amount**
 - **Total bill after tax**

Example Input

Units Consumed: **250**

Output

Total Bill (before tax): **PKR 1,750**

Total Tax: **PKR 175**

Total Bill (after tax): **PKR 1,925**

Example Input

Units Consumed: **350**

Output

Total Bill (before tax): **PKR 8,540**

Total Tax: **PKR 854**

Total Bill (after tax): **PKR 9,394**

Question 3

Advanced Health Insurance Premium Calculator

Develop a program that calculates **health insurance premiums** based on the following criteria:

Age of the applicant:

- **Under 18:** 2,000 PKR per month
- **18 – 30:** 3,500 PKR per month
- **31 – 50:** 5,000 PKR per month
- **Above 50:** 8,000 PKR per month

Pre-existing conditions:

- **No pre-existing conditions:** no additional charge
- **One pre-existing condition:** 25% increase in premium
- **More than one pre-existing condition:** 50% increase in premium

Smoker status:

- An additional **20%** if the applicant is a **smoker**.

The program should ask the user for their **age**, whether they have **pre-existing conditions**, and if they **smoke**. It should then calculate and display the **total premium**, including all adjustments.

Ensure **input validation** is present for **age and condition status**.

Question 4

Create a C++ program that assesses an individual's **qualification for a personal loan** based on the following criteria:

- **Annual income:** Must be at least **1,500,000 PKR**.
- **Credit score:** Must be **above 700**.
- **Debt-to-income ratio:** Must not exceed **35%**.
- **Employment status:** Must be **employed for at least the last 2 years**.

The program should ask for these inputs and use a series of **nested if-else statements** to determine:

- If the applicant is **eligible for the loan**.
- If not, provide a **specific reason** (e.g., income too low, credit score too low, high debt-to-income ratio, or insufficient employment duration).

Make sure that once there is enough information that **deems the user unqualified for a loan** the system must **no longer take further inputs** and stop and inform the user that they are **ineligible**.

Question 5

Secure Login System with Multi-Layer Authentication

Design a program that simulates a **multi-layer authentication system for a secure login**.

A user must input:

- A **username**
- A **password**

- A **2-factor authentication code** (e.g., a **6-digit OTP**)

The program should allow access if:

1. The **password is correct**.
2. When the **username and password match**, the **2-factor authentication code** is entered correctly.
3. The user does **not have any previous failed login attempts in the current session**.

If any of these conditions fail, display an **appropriate error message**.

Hint: Use the **NOT operator** to check if there are **no failed login attempts** (!failedAttempts), and combine conditions for the password and OTP.

Username, password, and OTP will all be of **int type**.

Question 6

Smart Energy Billing System

A smart energy billing system is developed to **optimize billing and load management based on the first digit of a user's 5-digit customer number**. The system analyzes the first digit and applies different scenarios to calculate the user's final bill.

- The user will input their **5-digit customer number** and **monthly power usage in kWh**.
 - The program will determine the scenario based on the **first (leftmost) digit** of the customer number.
-

Load Management Scenarios

1. If the first digit is a prime number (2, 3, 5, 7):

- The system will add a **10% discount** on the total bill if the user's usage is **less than 300 kWh**.
- If the usage is **above 300 kWh**, the customer receives a **flat rate bill with no discount**.

2. If the first digit is an odd number but not prime (1, 9):

- The user will be charged **extra for peak-hour usage**.
 - The system will ask the user to input their **peak-hour consumption in kWh**.
 - If **peak-hour consumption is more than 50%** of the total monthly usage, a **15% surcharge** is applied on the total bill.
 - Otherwise, the user pays the **normal rate with no surcharge**.
-

3. If the first digit is 0 or an even number (4, 6, 8):

- The user is classified as a **low-risk consumer** and is **exempt from all surcharges**.
 - They will only be charged a **flat rate** for their monthly usage, with **no discounts or extra charges**.
-

Input/Output Example

Example 1

Input:

Customer Number: **35792**

Monthly Usage: **280 kWh**

The first digit is **3**, which is a **prime number**.

Since the usage is **less than 300 kWh**, the user receives a **10% discount**.

Output:

10% discount applied. Final bill: X

Example 2

Input:

Customer Number: **91234**

Monthly Usage: **500 kWh**

The first digit is **9**, which is an **odd number but not prime**.

The system asks for **peak-hour usage: 300 kWh**.

Since peak-hour consumption is **60% of the total**, a **15% surcharge** is applied.

Output:

15% surcharge applied. Final bill: Y

Example 3

Input:

Customer Number: **40856**

Monthly Usage: **600 kWh**

The first digit is **4**, which is an **even number**.

The user is classified as **low-risk** and pays the **flat rate with no adjustments**.

Output:

No discounts or surcharges. Final bill: Z

You need to use **switch statements** for scenarios.

Question 7

Rightmost Digit Manipulation

You are tasked with creating a program that takes a **6-digit number** as input from the user and repeatedly modifies it based on the **rightmost digit** until **only one digit remains**.

The rightmost digit is **discarded after each step**, and the process continues with the remaining number.

Process

1. The user will input a **6-digit number**.
2. The program checks the **rightmost digit** of the number:
 - If the last digit is **even**, add **1** to the remaining number (after discarding the last digit).

- If the last digit is **odd**, subtract **1** from the remaining number (after discarding the last digit).
 - If **adding 1 results in the remaining number gaining an extra digit**, subtract **2 instead**.
3. Discard the **rightmost digit** and apply the same process to the new modified number.
 4. Repeat this process until **only 1 digit remains**.
 5. The program will then **print the final digit**.
-

Input/Output Examples

Example 1

Input:

999992

Final result:

8

Example 2

Input:

123456

Final result:

0

Question 8

Trip Planner for Weather Conditions

Design a **trip planner for outdoor activities**.

The user inputs:

- **Weather forecast:** sunny, rainy, snowy, or foggy

- **Temperature** in degrees Celsius
- **Wind speed** in km/h

The program should output **whether the trip is allowed or not**.

The trip is **not allowed** if:

- The weather is **not sunny or clear**.
- The **temperature is below 10°C or above 40°C**.
- The **wind speed exceeds 50 km/h**.

If any of these conditions are met, the program should **advise against the trip**.

If all conditions are met, display a message **allowing the trip**.

Hint: Use the **NOT operator** to negate the safe weather condition and check for extremes in temperature and wind speed.

Question 9

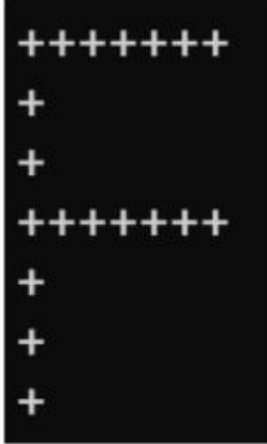
Display English alphabets

Implement a C++ program that displays each letter of the English alphabet as an ASCII art representation, adhering to the following guidelines:

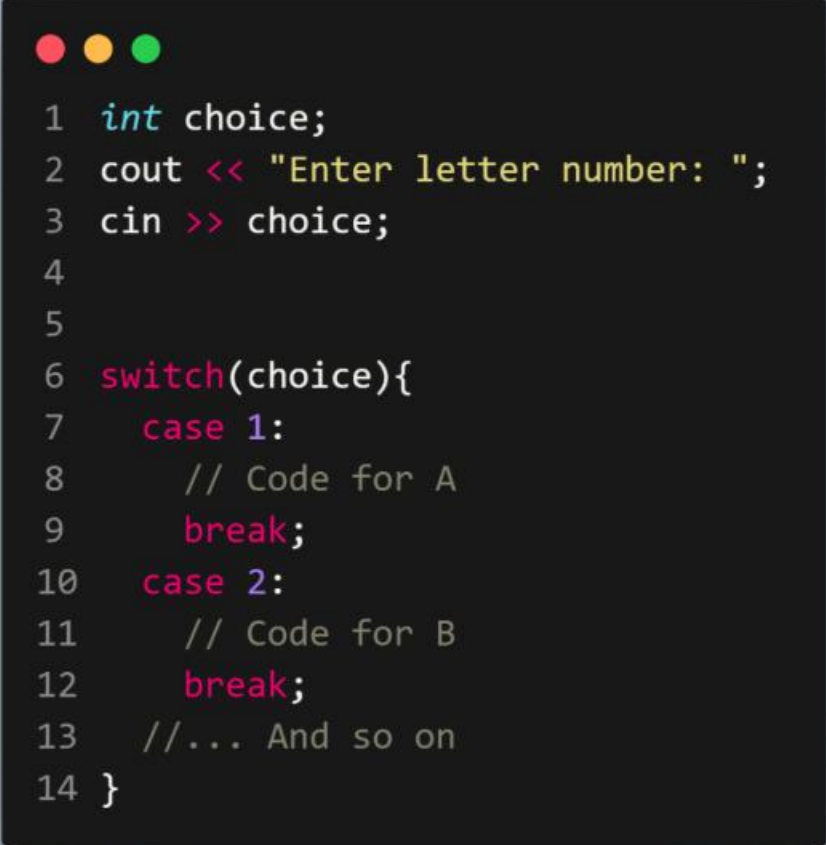
- Each letter should fit within a maximum width of 12 columns (including spaces) and a height of 7 rows.
- Use switch statements to determine the pattern for each letter and nested for loops to print the characters accordingly. Look at the end of the document to see the structure.
- Make effective use of loops and conditional statements to construct letters **C,E,F,H,I,L,O,T,U,V** ensuring no hardcoding of individual characters. **Hardcoded solutions will receive zero marks.**

Example Outputs (Attached):

Note: These outputs are provided for clarification. You are required to implement only the 10 characters mentioned above.

	<pre> for (int i = 1; i <= 5; i++) { for (int j = 1; j <= 9; j++) { if ((j == 6 - i) (i == 3 && j >= 3 && j <= 7) (j == 4 + i)) cout << "+"; else cout << " "; } cout << endl; } </pre>
	<pre> for (int i = 1; i <= 7; i++) { for (int j = 1; j <= 7; j++) { if (j == 1 i == 1 i == 4) cout << "+"; else cout << " "; } cout << endl; } </pre>
	<pre> for (int i = 1; i <= 7; i++) { for (int j = 1; j <= 7; j++) { if (j == 4 i == 1 i == 7) cout << "+"; else cout << " "; } cout << endl; } </pre>

Sample Switch Structure:



```
1  int choice;
2  cout << "Enter letter number: ";
3  cin >> choice;
4
5
6  switch(choice){
7      case 1:
8          // Code for A
9          break;
10     case 2:
11         // Code for B
12         break;
13     //... And so on
14 }
```